Les algorithmes (1/2) : chercher, mesurer

Fiche de lecture — recherche linéaire et dichotomique, notation grand O

Kevin Bardoux

Table des matières

1. Introduction 2
   Un tableau vu par l'ordinateur 2
2. La recherche 2
   La recherche linéaire 2
   La recherche dichotomique 3
   Un exemple pas à pas 3
3. L'efficacité des algorithmes 4
   Compter les étapes 4
   La notation grand O 5
   Pourquoi c'est si important 5
   Grand O, oméga et thêta 6
4. Regrouper des données : les objets 6
Sources 7

> Note
>
> **Lecture facultative**, environ 15 minutes, en complément de « Recherche, tris & complexité » (semaine 3, jeudi). Suite : Les algorithmes (2/2) : trier, récursivité. D'après CS50, Harvard University, licence CC BY-NC-SA 4.0, leçon 3 (CS50 2026) ; adaptation de Kevin Bardoux, code traduit en JavaScript. Cette fiche est diffusée sous la même licence.

1. Introduction

En semaine 2, on a vu qu'on trouve un nom dans un annuaire bien plus vite en le coupant en deux qu'en tournant les pages une à une. On formalise ici ces idées : chercher une valeur dans un tableau, et mesurer l'efficacité d'un algorithme.

Un tableau vu par l'ordinateur

Pour un humain, un tableau se voit d'un coup d'œil. Pour l'ordinateur, c'est une rangée de **casiers fermés** : il ne peut ouvrir qu'un casier à la fois.

Index	0	1	2	3	4	5	6
Casier	?	?	?	?	?	?	?

La question devient : dans quel ordre ouvrir les casiers pour trouver une valeur le plus vite possible ? **Entrée** : un tableau et une valeur à chercher. **Sortie** : vrai ou faux (trouvé ou non), ou bien la position de la valeur.

2. La recherche

La recherche linéaire

On ouvre les casiers un par un, de gauche à droite.

```
Pour chaque case, de la première à la dernière
  Si la case contient la valeur cherchée
    Renvoyer vrai
Renvoyer faux
```

Attention au piège : « renvoyer faux » se place **après** la boucle. Si on l'écrit dans un `else` à l'intérieur, on abandonne dès la première case différente !

```
/** Returns the index of value in numbers, or -1. */
function linearSearch(numbers, value) {
  for (let i = 0; i < numbers.length; i++) {
    if (numbers[i] === value) {
      return i;
    }
```

```
  }
  return -1; // after the loop: every box has been
opened
}

linearSearch([20, 500, 10, 5, 100, 1, 50], 100); // 4
```

Pour chercher un texte, `===` suffit en JavaScript : `'Chloé' === 'Chloé'` vaut `true` (en C, il faut une fonction dédiée, `strcmp`).

La recherche dichotomique

Si le tableau est **trié**, on peut faire beaucoup mieux, comme avec l'annuaire :

```
S'il ne reste plus de case
  Renvoyer faux
Regarder la case du milieu
Si elle contient la valeur cherchée
  Renvoyer vrai
Sinon si la valeur cherchée est plus petite
  Chercher dans la moitié gauche
Sinon si la valeur cherchée est plus grande
  Chercher dans la moitié droite
```

À chaque étape, on élimine **la moitié** des cases restantes.

Un exemple pas à pas

On cherche 50 dans le tableau trié :

Index	0	1	2	3	4	5	6
Valeur	1	5	10	20	50	100	500

Étape	Zone de recherche	Milieu	Valeur au milieu	Décision
1	index 0 à 6	3	20	50 est plus grand : à droite

Étape	Zone de recherche	Milieu	Valeur au milieu	Décision
2	index 4 à 6	5	100	50 est plus petit : à gauche
3	index 4	4	50	Trouvé

Trois étapes au lieu de cinq pour la recherche linéaire. L'écart se creuse énormément avec de grands tableaux.

```
/** Returns the index of target in the SORTED array, or
-1. */
function binarySearch(array, target) {
  let left = 0;
  let right = array.length - 1;
  while (left <= right) {
    const middle = Math.floor((left + right) / 2);
    if (array[middle] === target) {
      return middle;
    } else if (array[middle] < target) {
      left = middle + 1;
    } else {
      right = middle - 1;
    }
  }
  return -1; // not found
}
```

3. L'efficacité des algorithmes

Compter les étapes

Combien de comparaisons faut-il, dans le pire des cas (valeur absente) ?

Taille	Linéaire	Dichotomie
10	10	4
1 000	1 000	10
1 000 000	1 000 000	20

Taille	Linéaire	Dichotomie
1 000 000 000	1 000 000 000	30

Pour un milliard de valeurs : **30 comparaisons** au lieu d'un milliard. Mais la dichotomie a un prix : le tableau doit être **trié**. Et trier prend du temps…

La notation grand O

Plutôt que de compter chaque étape, les informaticiens s'intéressent à l'**ordre de grandeur** : comment le temps évolue quand la taille n des données augmente.

Notation	Nom	Exemple
$O(1)$	Constant	Lire une case d'un tableau
$O(\log n)$	Logarithmique	Recherche dichotomique
$O(n)$	Linéaire	Recherche linéaire
$O(n \log n)$	Quasi linéaire	Tri par fusion
$O(n^2)$	Quadratique	Tri par sélection, tri à bulles

On ignore les constantes et les termes plus petits : $n/2$ ou $3n + 5$, c'est toujours $O(n)$, car pour de très grands n, c'est la **forme de la courbe** qui compte.

Pourquoi c'est si important

Nombre d'opérations selon la taille des données :

n	$\log n$	n	$n \log n$	n^2
8	3	8	24	64
1 000	10	1 000	10 000	1 000 000
1 000 000	20	1 000 000	20 000 000	1 000 000 000 000

À raison d'un milliard d'opérations par seconde, trier un million d'éléments en $O(n^2)$ prend environ **17 minutes** ; en $O(n \log n)$, **2 centièmes de seconde**.

Grand O, oméga et thêta

Notation	Signification	Recherche linéaire	Recherche dichotomique
O (grand O)	Borne supérieure : le pire des cas	O(*n*)	O(log *n*)
Ω (oméga)	Borne inférieure : le meilleur des cas	Ω(1)	Ω(1)
Θ (thêta)	Quand O et Ω sont identiques	—	—

Dans le meilleur des cas, la valeur est dans la première case regardée : une seule étape, Ω(1). On utilise Θ quand le temps est toujours du même ordre, quel que soit le cas : par exemple, compter les éléments d'un tableau prend toujours Θ(*n*).

4. Regrouper des données : les objets

Un annuaire associe un nom à un numéro. Deux tableaux séparés (`names` et `numbers`) seraient fragiles : un décalage, et tout est faux. On regroupe plutôt les deux dans un **objet** (semaine 6) (MDN Web Docs, s. d.) ; en C, on écrirait une structure (`struct`).

```
const directory = [
  { name: 'Alice', number: '01 23 45 67 89' },
  { name: 'Bob', number: '06 12 34 56 78' },
];

function findNumber(directory, name) {
  for (const person of directory) {
    if (person.name === name) return person.number;
  }
  return null;
}
```

Le numéro est un **texte** et non un nombre : il commence par `0`, contient des espaces, et on ne fait jamais de calcul dessus. Le point (`.`) donne accès à une propriété.

Sources

CS50. 2026. « Lecture 3 - CS50x 2026 ». https://cs50.harvard.edu/x/notes/3/.

MDN Web Docs. s. d. « Utiliser les objets - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Working_with_objects.